

Angular Lifecycle Hooks Explained



Understand What Happens
Behind the Scenes

Learn how Angular components are
created, **updated**, **checked**, and **destroyed**.



CREATED

Component
is created



UPDATED

Component
is updated



CHECKED

Component
is checked



DESTROYED

Component
is destroyed



Save this post for future
Angular projects!



Better understanding.
Better **Angular** apps.

What Are Lifecycle Hooks?

Angular's Component Timeline

i

Lifecycle Hooks are special methods Angular calls at **different stages** of a **component's life**.



Component Created

Angular creates the component and initializes its properties.



Component Updated

Angular updates the component when data, inputs, or state changes.



Component Destroyed

Angular destroys the component and cleans up resources.



Hooks let you run code **at the right moment**.



ngOnChanges()



Detect Input Changes



TypeScript

```
1  ngOnChanges() {  
2      console.log('Input changed');  
3  }  
4
```



Runs when
@Input
changes



Executes
before
ngOnInit



React to
parent data
updates



Use **ngOnChanges()** to **respond** to changes in **@Input** properties.



ngOnInit()

Initialize Your Component



TypeScript

```
1  ngOnInit() {  
2      this.loadUsers();  
3  }  
4
```



Runs once
after component
creation



Common place
for API calls



Set initial
data



Most frequently
used lifecycle
hook.



Use **ngOnInit()** to **initialize** your
component and **load the data** you need.



ngDoCheck()

Custom Change Detection



TypeScript

```
1 ngDoCheck() {  
2   console.log('Checking changes');  
3 }  
4
```



Runs during
every change
detection cycle



Useful for
custom
tracking



Can affect
performance
if overused



Use **ngDoCheck()** only when you
need **custom change detection** logic.



ngAfterContentInit()

Projected Content Is Ready



TypeScript

```
1  ngAfterContentInit() {  
2    console.log('Content initialized');  
3  }  
4
```



Runs once
after content
projection



Works with
<ng-content>



Useful when using
projected content.



Use **ngAfterContentInit()** to
work with **content** passed from
parent components via **<ng-content>**.

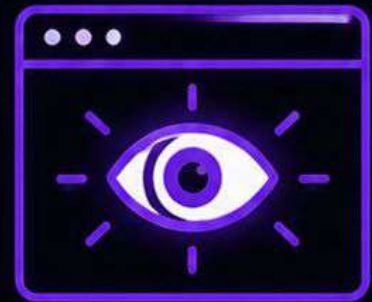


ngAfterViewInit()

View Is Fully Loaded

TypeScript

```
1 ngAfterViewInit() {  
2   console.log('View ready');  
3 }  
4
```



Runs after
component view
is initialized



Access
ViewChild
safely



Manipulate
DOM
elements



Perfect for
view-related
setup



Use **ngAfterViewInit()** when you need
the view and its children to be ready.



ngAfterViewChecked()



View Has Been Checked



TypeScript

```
1 ngAfterViewChecked() {  
2   console.log('View checked');  
3 }  
4
```



Runs after
every view check



Detect
view updates



Avoid
heavy operations
here



Use **ngAfterViewChecked()** to react
to changes in the view after checks.



ngOnDestroy()



Clean Up Resources

TypeScript

```
1  ngOnDestroy() {  
2      this.subscription.unsubscribe();  
3  }  
4
```



Runs before
component
removal



Unsubscribe
Observables



Remove
event
listeners



Prevent
memory
leaks



Always use **ngOnDestroy()** to **clean up** resources and **prevent memory leaks**.



Quick Recap ⚡

✓ ngOnChanges 

✓ ngOnInit 

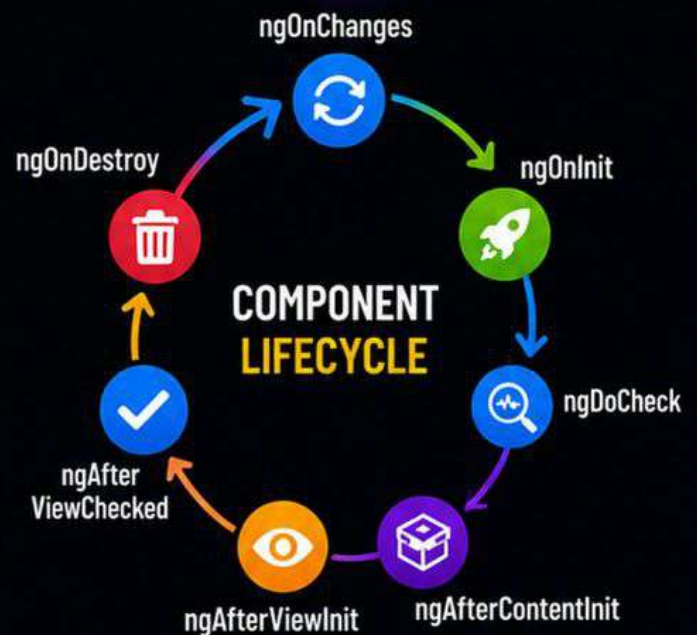
✓ ngDoCheck 

✓ ngAfterContentInit 

✓ ngAfterViewInit 

✓ ngAfterViewChecked 

✓ ngOnDestroy 



Save this post
for future Angular projects!

